

Competitive Programming

Section 28 Editorial: DP I Mixed Contest

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Training Schedule	4
2.1	Restatement	4
2.2	Constraints and traps	4
2.3	Observations	4
2.4	Algorithm	4
2.5	Correctness proof	5
2.6	Complexity	5
2.7	Full C++ solution	5
2.8	Full Python solution	5
2.9	Common wrong answers	6
2.10	Tests that matter	6
3	B. Two Carts	7
3.1	Restatement	7
3.2	Constraints and traps	7
3.3	Observations	7
3.4	Algorithm	7
3.5	Correctness proof	8
3.6	Complexity	8
3.7	Full C++ solution	8
3.8	Full Python solution	9
3.9	Common wrong answers	9
3.10	Tests that matter	10
4	C. Prerequisite Tree	11
4.1	Restatement	11
4.2	Constraints and traps	11
4.3	Observations	11
4.4	Algorithm	11
4.5	Correctness proof	12
4.6	Complexity	12
4.7	Full C++ solution	12
4.8	Full Python solution	13
4.9	Common wrong answers	14
4.10	Tests that matter	14
5	Testing discussion	15
6	What to carry forward	16

1 Editorial overview

Problem	Main idea	State
A. Training Schedule	previous action DP	<code>dp[last]</code>
B. Two Carts	2D 0/1 knapsack	<code>dp[a][b]</code>
C. Prerequisite Tree	tree knapsack	<code>dp[u][budget]</code>

2 A. Training Schedule

2.1 Restatement

Each day has zero, one, or two available activities. You can do at most one activity per day, and cannot repeat the same activity on consecutive days. Minimize rest days.

2.2 Constraints and traps

n is large, so trying all schedules is impossible. The only memory needed from the past is yesterday's activity.

The value 3 means both activities are available, not a third activity.

2.3 Observations

If yesterday's action is known, today's legal actions are determined:

```
rest is always legal
C++ is legal if available and yesterday was not C++
Python is legal if available and yesterday was not Python
```

2.4 Algorithm

```
dp[rest] = 0
dp[cpp] = INF
dp[python] = INF

for each day:
    next[rest] = min(dp) + 1
    if C++ available:
        next[cpp] = min(dp[rest], dp[python])
    if Python available:
        next[python] = min(dp[rest], dp[cpp])
    dp = next

print min(dp)
```

2.5 Correctness proof

Invariant. After processing a prefix of days, `dp[last]` is the minimum number of rest days among valid schedules for that prefix whose last action is `last`.

The empty prefix has cost 0 with previous action treated as rest. For a new day, every valid schedule either rests, does C++, or does Python. The transition checks exactly the legal previous actions for each choice and adds one rest day only in the rest case. Therefore it keeps the minimum cost for each possible last action. After all days, the best final last action gives the answer. $\square$

2.6 Complexity

```
Time:  O(n)
Memory: O(1)
```

2.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    const int INF = 1e9;
    array<int, 3> dp = {0, INF, INF};
    for (int day = 0; day < n; day++) {
        int available;
        cin >> available;
        array<int, 3> ndp = {min({dp[0], dp[1], dp[2]}) + 1, INF, INF};
        if (available & 1) ndp[1] = min(dp[0], dp[2]);
        if (available & 2) ndp[2] = min(dp[0], dp[1]);
        dp = ndp;
    }
    cout << min({dp[0], dp[1], dp[2]}) << '\n';
    return 0;
}
```

2.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    days = data[1 : 1 + n]
    inf = 10**9
    dp = [0, inf, inf]
    for available in days:
        ndp = [min(dp) + 1, inf, inf]
        if available & 1:
            ndp[1] = min(dp[0], dp[2])
        if available & 2:
            ndp[2] = min(dp[0], dp[1])
        dp = ndp
    print(min(dp))

if __name__ == "__main__":
    main()

```

2.9 Common wrong answers

Treating value 3 as a separate activity.
 Allowing the same activity on consecutive days.
 Forgetting that resting resets the previous activity.

2.10 Tests that matter

All days unavailable.
 All days only one same activity.
 Alternating forced activities.
 Days where both activities are available.

3 B. Two Carts

3.1 Restatement

Each item can go in cart A, cart B, or be skipped. Each cart has its own weight capacity. Maximize total value.

3.2 Constraints and traps

This is not ordinary one-capacity knapsack. A state must remember both used capacities.

Each item can be used at most once. Updating capacities in increasing order can reuse the same item.

3.3 Observations

After processing some items, the future only depends on:

```
used weight in cart A
used weight in cart B
best value achieved with those weights
```

3.4 Algorithm

```
dp[a][b] = best value with a weight in A and b weight in B

for each item (w, v):
    for a from A down to 0:
        for b from B down to 0:
            put in A if possible
            put in B if possible

answer = max over all dp[a][b]
```

3.5 Correctness proof

Invariant. After processing the first i items, $dp[a][b]$ is the maximum value obtainable using exactly a weight in cart A and b weight in cart B.

For item $i+1$, any optimal solution either skips it, puts it in A, or puts it in B. Skipping is represented by keeping the old value. The two update cases represent placing the item into a cart if capacity allows. Decreasing iteration ensures each update uses a state from before the current item was placed, so the item cannot be counted twice. Thus the invariant is preserved. Taking the maximum over capacities gives the best value with capacity at most the limits.

3.6 Complexity

Time: $O(n A B)$

Memory: $O(A B)$

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, cap_a, cap_b;
    if (!(cin >> n >> cap_a >> cap_b)) return 0;
    vector<pair<int, long long>> items(n);
    for (auto &[w, v] : items) cin >> w >> v;

    vector<vector<long long>> dp(cap_a + 1, vector<long long>(cap_b + 1, 0));
    for (auto [weight, value] : items) {
        for (int a = cap_a; a >= 0; a--) {
            for (int b = cap_b; b >= 0; b--) {
                long long cur = dp[a][b];
                if (a + weight <= cap_a) {
                    dp[a + weight][b] = max(dp[a + weight][b], cur + value);
                }
                if (b + weight <= cap_b) {
                    dp[a][b + weight] = max(dp[a][b + weight], cur + value);
                }
            }
        }
    }
}
```

```

long long answer = 0;
for (int a = 0; a <= cap_a; a++) {
    for (int b = 0; b <= cap_b; b++) answer = max(answer, dp[a][b]);
}
cout << answer << '\n';
return 0;
}

```

3.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    cap_a = next(it)
    cap_b = next(it)
    items = [(next(it), next(it)) for _ in range(n)]
    dp = [[0] * (cap_b + 1) for _ in range(cap_a + 1)]
    for weight, value in items:
        for a in range(cap_a, -1, -1):
            for b in range(cap_b, -1, -1):
                cur = dp[a][b]
                if a + weight <= cap_a:
                    dp[a + weight][b] = max(dp[a + weight][b], cur + value)
                if b + weight <= cap_b:
                    dp[a][b + weight] = max(dp[a][b + weight], cur + value)
    print(max(max(row) for row in dp))

if __name__ == "__main__":
    main()

```

3.9 Common wrong answers

Using one capacity equal to A+B; that loses which cart has space.
 Updating capacities upward and reusing one item.
 Forgetting the skip option.
 Returning only dp[A][B] when the best solution may leave unused capacity.

3.10 Tests that matter

An item too heavy for either cart.
Two items each exactly fill a cart.
Best answer leaves unused capacity.
High values that require 64-bit sums.

4 C. Prerequisite Tree

4.1 Restatement

Lessons form a rooted tree. Selecting a lesson requires selecting its parent. Each lesson has time and value. Choose lessons with total time at most B and maximum total value.

4.2 Constraints and traps

The tree condition means a child cannot be selected independently. Ordinary knapsack over all lessons would allow invalid choices.

The root does not have to be selected; choosing no lesson is valid and has value 0.

4.3 Observations

If node u is selected, then each child subtree can either contribute nothing or contribute a valid selected set rooted at that child.

This is exactly a knapsack merge over children.

4.4 Algorithm

```
build children lists
build postorder from root

for u in postorder:
    dp[u][time[u]] = value[u]
    for child v:
        merged = dp[u]
        for used budget in u's current table:
            for add budget in child's table:
                merged[used + add] =
                    max(merged[used + add], dp[u][used] + dp[v][add])
        dp[u] = merged

answer = max(0, max_b dp[root][b])
```

4.5 Correctness proof

Invariant. After processing some children of u , $dp[u][b]$ is the maximum value of a valid prerequisite-respecting selected set inside u 's subtree, using exactly b time, requiring that u is selected, and using only the already processed child subtrees.

Initialization with only u selected is correct: time $time[u]$, value $value[u]$.

When merging a child v , a valid solution either takes nothing from v , which is kept by copying $dp[u]$, or takes a valid set from v 's subtree. Because $dp[v][add]$ requires that v is selected, and u is already selected, the prerequisite edge from v to u is satisfied. Trying all budget splits and taking the maximum gives the best merged table.

By postorder, every child table is correct before it is merged. After all children are processed, the invariant gives the correct table for u . Applying this to the root and taking the best budget at most B , plus the option of choosing nothing, gives the answer. $\square$

4.6 Complexity

Time: $O(n B^2)$

Memory: $O(n B)$

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, budget;
    if (!(cin >> n >> budget)) return 0;
    vector<int> need(n);
    vector<long long> value(n);
    for (int i = 0; i < n; i++) cin >> need[i] >> value[i];
    vector<vector<int>> children(n);
    for (int child = 1; child < n; child++) {
        int parent;
        cin >> parent;
        children[parent - 1].push_back(child);
    }

    vector<int> order = {0};
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : children[u]) order.push_back(v);
    }
}
```

```

}

const long long NEG = -(1LL << 60);
vector<vector<long long>> dp(n, vector<long long>(budget + 1, NEG));
for (int idx = n - 1; idx >= 0; idx--) {
    int u = order[idx];
    if (need[u] <= budget) dp[u][need[u]] = value[u];
    for (int v : children[u]) {
        vector<long long> merged = dp[u];
        for (int used = 0; used <= budget; used++) {
            if (dp[u][used] == NEG) continue;
            for (int add = 1; used + add <= budget; add++) {
                if (dp[v][add] == NEG) continue;
                merged[used + add] = max(merged[used + add], dp[u][used] + dp[v][add]);
            }
        }
        dp[u].swap(merged);
    }
}

long long answer = 0;
for (long long x : dp[0]) answer = max(answer, x);
cout << answer << '\n';
return 0;
}

```

4.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    budget = next(it)
    time = [0] * n
    value = [0] * n
    for i in range(n):
        time[i] = next(it)
        value[i] = next(it)
    children = [[] for _ in range(n)]
    for child in range(1, n):
        parent = next(it) - 1
        children[parent].append(child)

```

```

order = [0]
for u in order:
    order.extend(children[u])

neg = -10**30
dp = [[neg] * (budget + 1) for _ in range(n)]
for u in reversed(order):
    if time[u] <= budget:
        dp[u][time[u]] = value[u]
    for v in children[u]:
        merged = dp[u][:]
        for used in range(budget + 1):
            if dp[u][used] <= neg // 2:
                continue
            for add in range(1, budget - used + 1):
                if dp[v][add] <= neg // 2:
                    continue
                candidate = dp[u][used] + dp[v][add]
                if candidate > merged[used + add]:
                    merged[used + add] = candidate
        dp[u] = merged
answer = max(0, max(dp[0]))
print(answer)

if __name__ == "__main__":
    main()

```

4.9 Common wrong answers

Choosing a child without its parent.
 Merging child tables before they are computed.
 Forgetting that taking nothing from a child is allowed.
 Returning only exact budget B instead of at most B.
 Failing when the root is too expensive and the answer should be 0.

4.10 Tests that matter

Single root too expensive.
 A chain where selecting the deepest node requires all ancestors.
 A star where child choices compete for budget.
 Best answer leaves unused budget.

5 Testing discussion

For this mixed contest, the random generators use brute force where practical:

```
schedule: direct DP cross-check  
two carts: try all  $3^n$  choices for small  $n$   
tree prerequisites: enumerate all subsets for small  $n$ 
```

That is the right habit for upsolving DP. First prove the transition, then stress it against brute force on tiny random cases.

6 What to carry forward

The main skill from this section is diagnostic:

```
If only the previous action matters, keep only the previous action.  
If two resources constrain the future, keep both.  
If choices live under a tree, solve children before parents.  
If a budget is attached to each subtree, merge budgets like knapsack.
```